

AT Curs 2

1. Grafuri

2. Parcurgerea grafurilor

3. BFS - algoritmi

- apel

- aplicații

↙ complexitate

det. arbore parțial

det. distanță

det. drum minim

4. DFS

Grafuri

Mod de reprezentare

- matrice de adiacență
- listă de adiacență
- listă de muchii

↳ alegere :

Memorie consumată pt. graf cu n vârfuri
 m muchii

mat. ad : $O(n^2)$

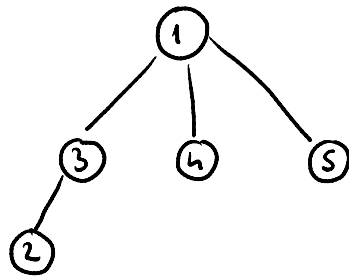
l. a. : $O(n + m)$

l. m. : $O(m)$

$$m \leq \frac{n(n-1)}{2}$$

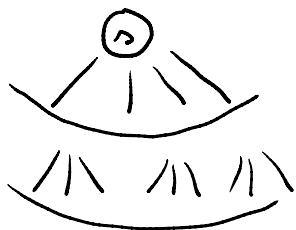
↳ pt un graf dens (aproape complet) m. a. nu
este mai "rea"

Parcurgerea Grafurilor



- vf de start
- folosind muchiile $\rightarrow$ vizităm nodurile accesibile din s într-o anumită ordine

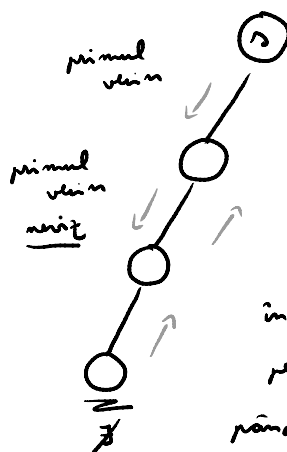
Lățime (BFS) $\rightarrow$ coadă



vecinii
vecinii vecinilor
(nepotăți)

$s = 1$ 1, 3, 4, 5, 2
 $s = 4$ 4, 1, 3, 5, 2

Adâncime (DFS)



înapoi pe drumul
pe care am venit
până la un vf
care mai are vecini
nervi

$s = 1$ 1, 3, 2, 4, 5

BFS

$$O(n) \leftarrow \text{BFS}(s)$$

initializare

$$O(1) \left\{ \begin{array}{l} \text{adauga } \ell \quad (O(1) \text{ adăugare / elim}) \\ \text{adauga } (\ell, s) \\ \text{viz}[s] = 1 \end{array} \right.$$

vector în care marcăm viz
vizitate ($\sum_{u \in V}$ memorăm ca listă
în vector $\rightarrow$ test $O(1)$ dacă
un u e vizitat

cat timp $\ell \neq \emptyset$

$$\begin{array}{l} \swarrow \\ \begin{array}{l} \text{m.a. } O(n) \\ \text{l.a. } O(\deg(u)) \\ \text{l.m. } O(m) \end{array} \end{array} \quad \begin{array}{l} u = \text{extrag}(\ell) \text{ pentru } v \text{ vecin al lui } u \\ \left\{ \begin{array}{l} \text{dacă } \text{viz}[v] = 0 \text{ (nerez)} \\ \text{adauga } (\ell, v) \\ \text{viz}[v] = 1 \rightarrow \text{marcat la adăugare} \end{array} \right. \\ \begin{array}{l} (*) \text{ tata}[v] = u \\ (*) d[v] = d[u] + 1 \end{array} \end{array}$$

$\Downarrow$

$$\text{m.a. } O(n^2)$$

$$\text{l.a. } O(n + \sum_{u \in V} \deg(u))$$

$\Downarrow$

$$O(n + m)$$

$$\text{l.m. } O(nm)$$

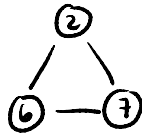
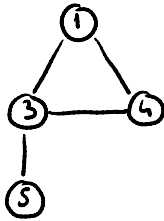
Apel

1) $BFS(s)$ - doar vf accesibile din s

2) pentru $v \in V$

dacă $viz[v] = 0$

$BFS(v) \rightarrow$ o componentă conexă



$BFS(1)$ 1, 3, 4, 5

$BFS(2)$ 2, 6, 7

Aplicații BFS

3) Testare conex

$BFS(1)$ + verificare toate nodurile
sunt viz

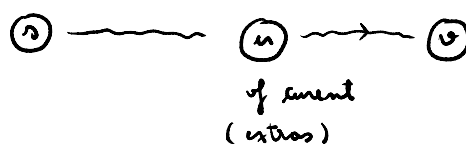
4) Dat un arbore parțial în graf conex

$\Rightarrow$ vector $tata_i$

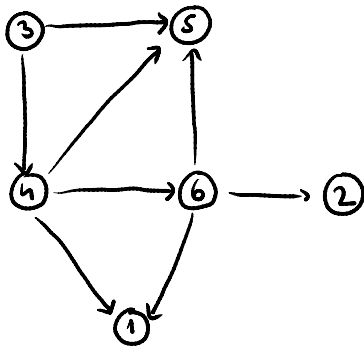
(*) $tata[v] = u$ ($tata$ vf-ului v este vf u
din care a fost descoperit

5) Dat distanțe de la s dat la toate
celelalte

(*) $d[v] = d[u] + 1$



6) Dat $u, v \Rightarrow$ det. distanța de la u la v și un drum minim

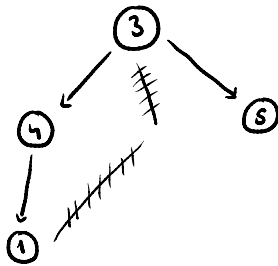


$$u = 3$$

$$v = 1$$

$BF(3) - \left\{ \begin{array}{l} \text{vet. tata} \rightarrow \text{pt a memora} \\ \text{drumul} \end{array} \right.$

Obs BF se poate opri cand e vizitat v
(drumul se obt. din vet. de tata pornind din v)



	1	2	3	4	5	6
tata	4		0	3	3	

$$u = 3$$

$$v = 1$$

$$v = 1 \Rightarrow \text{tata}[v] = 4 \Rightarrow \text{tata}[4] = 3$$

$$\left[\overset{v}{1}, \overset{u}{4}, 3 \right] \rightarrow \text{trebuie inversat}$$

$$\text{bont}(x, u)$$

$$\text{while}(x \neq u)$$

??

si se x

$$x = \text{tata}[x]$$

DFS

